Contract Risk Red-Flagger — Project Guide

Nov 14, 2026 · @Kavya

Overview

A user uploads a contract and gets back the clauses worth worrying about: the exact text, a plain-English explanation, a low/medium/high risk rating, fairer wording to ask for, and warnings for important clauses that are missing.

For each clause type, the pipeline runs five steps:

1. **Search.** Cut the contract into ~250-word chunks and find the 5 most relevant for that clause type (sentence-transformers `all-MiniLM-L6-v2`).
2. **Find.** An LLM reads those 5 chunks and either quotes the exact clause or says it isn't there.
3. **Check.** Code confirms the quote appears word for word in the contract. Anything that doesn't is thrown out.
4. **Explain, rate, rewrite.** The LLM explains the clause, rates its risk against our written rubric, and suggests fairer wording.
5. **Show.** A FastAPI backend and a simple upload page list the clauses, colour-coded by risk.

We then prove it works by comparing it on 100 held-out contracts against two simpler systems: a model trained on CUAD's labels, and an LLM given the whole contract in one prompt.

Why an LLM and not just a trained model: CUAD's labels only say *where* a clause is. They say nothing about whether it's risky, what it means, or how to fix it. The trained model is our baseline for finding clauses; the LLM does the reading and reasoning.

Clause types

We cover 15 clause types: every type at least one of us voted for. "Contracts" is how many of CUAD's 510 contracts contain the clause.

Clause type (CUAD name)	Kavya	Arnav	Sophia	Contracts	% of 510
Non-Compete	1	1	1	119	23.3
Ip Ownership Assignment	1	1	1	124	24.3
Uncapped Liability	1	1	1	111	21.8
Renewal Term	1	1	1	176	34.5

Clause type (CUAD name)	Kavya	Arnav	Sophia	Contracts	% of 510
Termination For Convenience	1	1	1	183	35.9
Exclusivity	1	1	0	180	35.3
Minimum Commitment	1	1	0	165	32.4
Notice Period To Terminate Renewal	0	1	1	111	21.8
Governing Law	0	1	1	437	85.7
Cap On Liability	0	1	1	275	53.9
Change Of Control	1	0	0	121	23.7
Post-Termination Services	1	0	0	182	35.7
Insurance	1	0	0	166	32.5
Anti-Assignment	0	0	1	374	73.3
Liquidated Damages	0	0	1	61	12.0

Things to keep in mind when reporting results:

- **Uneven spread.** Governing Law (86%), Anti-Assignment (73%) and Cap On Liability (54%) appear in most contracts, so "missing" is rare and recall looks easy. Liquidated Damages (12%) has only ~12 examples in the 100-contract test set, so its scores will be noisy. Report every metric per clause type, not just one average.
- **Linked pairs.** Notice Period To Terminate Renewal only exists alongside Renewal Term. Cap On Liability and Uncapped Liability are two sides of the same question. Treat each pair together in the risk rubric so the ratings don't contradict each other.
- **Governing Law** is rarely risky on its own. Its risk depends on whose state it is, so the rubric needs to say how to judge that.

Where we are

Setup, data and a first retrieval test are done. The search works well for most of the original 10 clause types and needs fixing for four.

Done	Detail
Repo	`kavyajain10/GenAI-Final-Project`, with `data/`, `notebooks/`, `src/`, `.gitignore`, `requirements.txt`
Environment	Python venv (`.venv`) inside the repo. Use `pip install -r requirements.txt`.

Done	Detail
Data	CUAD v1 (`CUADv1.json` , 510 contracts) in `data/` . Not pushed to GitHub; download it from the CUAD repo.
Split	`data/split.json` : 410 dev, 100 test (seed 42). The test set stays untouched until the final evaluation.
Chunking	`src/chunking.py` : ~250-word chunks along paragraph breaks. 95% of labelled clauses fit inside one chunk. Each chunk keeps its character positions in the original text.
Retrieval test	`notebooks/02_retrieval.ipynb` : checks whether the right chunk lands in the top 1/3/5/10 search results.

First retrieval scores (top 5, 50 dev contracts, original 10 types): average 0.80. Fine (≥0.89): Termination For Convenience, Insurance, Renewal Term, Uncapped Liability, Post-Termination Services, Change Of Control. Need work: Non-Compete 0.36, Exclusivity 0.65, Minimum Commitment 0.71, Ip Ownership Assignment 0.79. The 5 newly added types haven't been tested yet.

Contracts are long: a median of ~5,000 words and a maximum of ~47,700. This matters for the whole-contract baseline.

Step-by-step plan

Twelve steps remain. Steps 1–8 build and score the pipeline on dev contracts; steps 9–12 compare it against the baselines, build the app and write it up.

1. **Finish retrieval.**
   - Run `02_retrieval.ipynb` with all 15 clause types on all 410 dev contracts.
   - For any type below 0.85 at top 5: reword its query to sound like the real clauses (the "misses" cell shows them), or raise its keyword boost.
   - Done when: every type scores ≥0.85 at top 5, or we've decided to send more chunks for the weak ones.
2. **Pick the LLM.**
   - A cheap API model is the default. Small local models struggle with legal text and fixed output formats.
   - Check whether the course provides API credits or expects a particular model.
   - Done when: we can call it from Python and get a response.

3. **Clause extraction.**
   - For each clause type, send the top 5 chunks and ask the LLM to return JSON: `{"found": true/false, "quote": "..."}`.
   - Done when: it runs on 20 dev contracts with no crashes or malformed answers.
4. **Quote check.**
   - Confirm every quote appears word for word in the contract. Normalise spaces, line breaks and curly quotes on both sides first.
   - Done when: a tested function `quote_is_real(quote, contract)` exists.
5. **Score extraction on dev.**
   - Per clause type: precision, recall, absent accuracy (correctly saying "not there") and made-up quote rate. A quote counts as correct if it overlaps CUAD's marked text.
   - Improve the prompt for the worst types. Never tune on the test set.
   - Done when: a results table exists and the prompt has been tuned at least once.
6. **Risk rubric** (can start now).
   - For each clause type, 1–2 sentences each for High / Medium / Low risk, from the small business's side.
   - Say which *missing* clauses matter (no Termination For Convenience = bad; no Non-Compete = good).
   - Done when: one page covers all 15 types.
7. **Hand-label risk.**
   - Rate ~50 real dev clauses (3–4 per type) using the rubric, before seeing any LLM ratings. Two people per clause if possible.
   - Done when: a sheet holds clause text, rating and reason.
8. **Explain, rate, rewrite.**
   - Second prompt. Input: the quote and that type's rubric. Output JSON: `summary`, `risk`, `reason`, `suggested_rewrite`. Plus flags for important missing clauses.
   - Done when: it runs on the 50 hand-labelled clauses.
9. **Score the risk analysis.**
   - Ratings: % agreement with our hand labels, and where they differ.
   - Explanations and rewrites: an LLM judge scores each 1–5; we grade ~20 ourselves to check the judge.
10. **Build the two baselines.**
    - Whole-contract LLM: the same extraction prompt on the full contract, no search. Long contracts need a long-context model.

- Trained model: TF-IDF + logistic regression per clause type, labelling each chunk "contains clause or not", trained on dev.
- Done when: both output found/not-found in the same format as our pipeline.

11. **Final evaluation on the 100 test contracts.**
    - Run all three systems and fill one table: precision, recall, absent accuracy, made-up quotes, cost and time per contract. Add the risk results from step 9.
12. **Web app, write-up, demo.**
    - FastAPI `/analyze` endpoint: upload a PDF or text file, extract the text (`pypdf`), run the pipeline, return JSON.
    - Simple page: clauses coloured red/yellow/green with quote, explanation and rewrite, plus missing-clause warnings.
    - Write-up: problem, method, data, results table, good and bad examples, limitations. Demo on 1–2 pre-tested contracts.

Who does what

The work splits into three streams that can run at the same time. Owners are still to be decided.

Stream	Steps	Owner
Search + extraction: retrieval, LLM extraction, quote check, extraction scoring	1, 3, 4, 5	
Risk: rubric, hand labels, risk prompt, LLM judge	6, 7, 8, 9	
Baselines + app: whole-contract LLM, trained model, FastAPI and the upload page	10, 12 (app)	
Everyone	2, 11, 12 (write-up, demo)	All

Next steps

These can all start now, in parallel.

- [ ] Pull the updated `02_retrieval.ipynb` (15 clause types, keyword boost) and run it on 50 dev contracts
- [ ] Switch it to all 410 dev contracts and record the scores
- [ ] Fix queries for any clause type below 0.85 at top 5
- [ ] Choose the LLM and get an API key working

- [ ] Start the risk rubric for all 15 clause types
- [ ] Update the proposal's clause list to these 15 types
- [ ] Assign owners for the three streams

Open decisions

- Which LLM, and who pays for API calls?
- What is the final deadline? Needed to set weekly dates.
- For the whole-contract baseline: use a long-context model, or skip/truncate contracts that don't fit?
- Trained baseline: stay with TF-IDF + logistic regression, or also fine-tune a transformer?
- Which missing clauses should trigger a warning?